PDF LOGO STAMPER • เวอร์ชันแอป 1.3.0

คู่มือติดตั้งบน Windows ด้วย Docker

สำหรับนำไปติดตั้งบนเครื่องอื่นเพื่อให้ทั้งทีมใช้งานร่วมกัน

จัดทำเมื่อ 14/10/2026 | ต้องใช้อย่างน้อย: Windows 10/11 (64-bit), Docker Desktop แบบ WSL2, RAM 4 GB, พื้นที่ดิสก์ว่าง 5 GB

ระบบจัดเก็บข้อมูลไว้ในเครื่องเท่านั้น ไม่ส่งออกนอกเครือข่าย

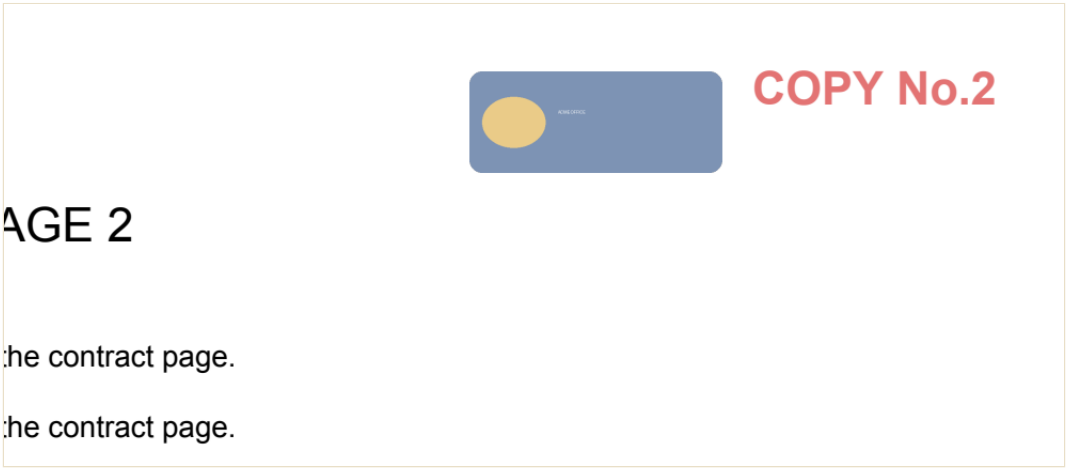
1. ระบบทำงานอย่างไร

PDF Logo Stamper คือเว็บแอปสำหรับ **วาง watermark (โลโก้ + ข้อความ เช่น "COPY No.1") ที่มุมขวาบนของ PDF ทุกหน้า** โดยไม่แก้ไขเนื้อหาต้นฉบับ รองรับหลายไฟล์พร้อมกัน สร้างสำเนาได้สูงสุด 200 ชุดต่อรอบ และบันทึกชุดการตั้งค่าไว้ใช้ซ้ำกับงานเดิมได้

ระบบรันด้วย Docker Compose ประกอบด้วย 2 คอนเทนเนอร์:

บริการ	หน้าที่	พอร์ต
frontend	หน้าจอเว็บสำหรับใช้งาน (nginx) — เปิดใช้งานจริงที่พอร์ตนี้	8090
backend	ระบบคำนวณและวาง watermark (FastAPI) — ทำงานภายในเท่านั้น	9010 (ภายใน) 9012 (เฉพาะเครื่องที่ติดตั้ง)
stamper_data (volume)	เก็บโลโก้ ไฟล์ PDF และงานที่บันทึกไว้ — ข้อมูลอยู่ต่อเมื่อรีสตาร์ทหรืออัปเดตเวอร์ชัน	—

ตัวอย่างผลลัพธ์จริง — โลโก้และข้อความสีแดงวางทับมุมขวาบน โปร่งจางจนอ่านเนื้อหาผ่านได้ (ปรับสี ขนาด ความจาง และความเอียงได้ที่หน้าจอ):



ตัวอย่าง watermark มุมขวาบนของเอกสาร สร้างด้วยระบบเดียวกับที่ใช้จริง

2. เตรียมเครื่อง Windows

2.1 สิ่งที่เครื่องต้องมี

หัวข้อ	ขั้นต่ำ	แนะนำ
ระบบปฏิบัติการ	Windows 10 64-bit (รุ่น 21H2 ขึ้นไป) หรือ Windows 11	Windows 11
หน่วยความจำ (RAM)	4 GB	8 GB
พื้นที่ดิสก์ว่าง	5 GB สำหรับ Docker Desktop และอิมเมจของแอป	SSD, 10 GB
อื่น ๆ	เปิดใช้งาน virtualization ใน BIOS (VT-x หรือ AMD-V)	—

2.2 ติดตั้ง WSL2 (ถ้ายังไม่มี)

เปิด **PowerShell (Administrator)** แล้วรันคำสั่งด้านล่าง (เครื่องอาจรีสตาร์ทเองหรือสั่งให้รีสตาร์ท แล้วกลับมาเปิด PowerShell ใหม่):

```
wsl --install
# reboot when prompted, then reopen PowerShell
wsl --version
```

2.3 ติดตั้ง Docker Desktop

- ดาวน์โหลดตัวติดตั้งจาก **docker.com/products/docker-desktop** แล้วติดตั้งตามขั้นตอน
- เปิด Docker Desktop และรอจนมุมล่างขวาของหน้าจอแสดง **“Engine running”** (ครั้งแรกอาจขอให้ Sign in — กดข้ามได้)
- ตรวจความพร้อมด้วย PowerShell:

```
docker --version
docker compose version
docker run --rm hello-world
```

ถ้าเห็นข้อความ “Hello from Docker!” แสดงว่าเครื่องพร้อมติดตั้งแอปแล้ว หากขึ้น error เรื่อง virtualization หรือ WSL ให้กลับไปทำข้อ 2.2

3. ดึงโค้ดจาก GitHub (public repository)

โค้ดเก็บไว้ที่ **github.com/nirut9999/pdf-logo-stamper** (public) — ดาวน์โหลดได้ทันที ไม่ต้องใช้บัญชี GitHub หรือขออนุญาต

บนเครื่อง Windows: ติดตั้ง **Git for Windows** จาก **git-scm.com** ก่อน (มาพร้อม Git Credential Manager) แล้วเปิด PowerShell:

```
cd C:\
git clone https://github.com/nirut9999/pdf-logo-stamper.git
cd pdf-logo-stamper
```

เมื่อ clone เสร็จจะได้โฟลเดอร์ `pdf-logo-stamper` พร้อมใช้ ครั้งถัดไปเมื่อต้องการรับเวอร์ชันใหม่ ก็เพียงรัน `git pull` ในโฟลเดอร์เดิม

> **หมายเหตุ:** กรณีเครื่องไม่มี Git หรือเน็ตถูกบล็อก ใช้วิธี ZIP จากเครื่องต้นทางแทน (`zip -r pdf-logo-stamper.zip pdf-logo-stamper -x "*/node_modules/*" "*/.git/*" "*/__pycache__/*" "*/dist/*"` แล้ว Extract All บน Windows) — ข้อมูลโลโก้/เอกสารผู้ใช้ไม่มากับทั้ง ZIP และ repo (เก็บในvolume ของเครื่องเดิม) ถ้าต้องการย้ายงานที่บันทึกไว้ไปด้วยให้ทำ backup ตามข้อ 7

4. ติดตั้งและเปิดใช้งานครั้งแรก

เปิด PowerShell แล้วรัน (ปรับ path ตามที่แตกไฟล์ไว้):

```
cd C:\pdf-logo-stamper
docker compose up -d --build
```

การ build ครั้งแรกต้องดาวน์โหลดฐาน Python/Node และไลบรารี ใช้เวลาประมาณ 3–10 นาที ตามความเร็วอินเทอร์เน็ต โดยมี log แสดงความคืบหน้าตลอด

เมื่อเสร็จให้ตรวจสถานะ — ทั้งสองตัวต้องเป็น Up (healthy):

```
docker compose ps
```

จากนั้นเปิดเบราว์เซอร์ที่ **http://localhost:8090** — ถ้าเห็นหน้าจอ PDF Logo Stamper แสดงว่าติดตั้งสำเร็จ ✓ ทดลองใช้งานจริง: อัปโหลดโลโก้ PNG → อัปโหลดไฟล์ PDF → ปรับค่า → ดูตัวอย่าง preview → กด Stamp & ดาวน์โหลด

5. ให้เครื่องอื่นในวง LAN ใช้ร่วมกัน

ดู IP ของเครื่องที่ติดตั้ง (PowerShell):

```
ipconfig | findstr IPv4
# e.g. 192.168.1.50
```

เปิด firewall ของ Windows สำหรับพอร์ต 8090 (PowerShell as Administrator):

```
New-NetFirewallRule -DisplayName "PDF Logo Stamper"
    -Direction Inbound -Protocol TCP -LocalPort 8090 -Action Allow
```

เครื่องอื่นในวง LAN เดียวกันเปิด `http://192.168.1.50:8090` ได้ทันที

> **หมายเหตุ:** แอปนี้ยังไม่มีระบบล็อกอิน — ควรเปิดให้เฉพาะวง LAN ที่ไว้ใจได้ หรือปรับ firewall ให้อนุญาตเฉพาะวง subnet ที่ใช้งานจริง

6. การหยุดและเริ่มระบบ

ต้องการ	คำสั่ง (รันในโฟลเดอร์โปรเจกต์)
เปิดใช้งานอีกครั้งหลังรีสตาร์ทเครื่อง	docker compose start
หยุดชั่วคราว (ข้อมูลอยู่ครบ)	docker compose stop
อัปเดตเป็นเวอร์ชันใหม่ (build ซ้ำ)	docker compose up -d --build
รีสตาร์ทเฉพาะฝั่ง backend	docker compose restart backend
ลบระบบและข้อมูลทั้งหมด	docker compose down -v ⚠ **ข้อมูลโลโก้ งานที่บันทึกไว้ และไฟล์ต่าง ๆ จะถูกลบถาวร**

ตั้งค่าให้ Docker Desktop เริ่มเองเมื่อลงชื่อเข้าเครื่อง: Docker Desktop → Settings → General → **“Start Docker Desktop when you sign in”** จากนั้นเพิ่มงานใน **Task Scheduler** ให้รัน `docker compose start` (Action: Start a program, Start in: ระบุโฟลเดอร์โปรเจกต์) เพื่อให้แอปพร้อมใช้ทุกครั้งหลังเปิดเครื่อง

7. สำรองข้อมูลและกู้คืน

โลโก้ ไฟล์ PDF ที่อัปโหลด และงานที่บันทึกไว้ (presets) เก็บใน volume ชื่อ `pdf-logo-stamper_stamper_data` — ชื่อจริงจะขึ้นต้นด้วยชื่อโฟลเดอร์โปรเจกต์ ตรวจได้ด้วยการรัน `docker volume ls` ข้อมูลไม่หายไปเมื่อรีสตาร์ทหรืออัปเดต แต่จะถูกลบเมื่อใช้ `down -v` จึงควรมีไฟล์สำรองเสมอ:

```
# backup -> stamper-backup.tgz in current folder
docker run --rm -v pdf-logo-stamper_stamper_data:/data -v ${PWD}:/backup `
    alpine tar czf /backup/stamper-backup.tgz -C /data .

# restore onto a new machine (before the first up -d is fine)
docker volume create pdf-logo-stamper_stamper_data
docker run --rm -v pdf-logo-stamper_stamper_data:/data -v ${PWD}:/backup `
    alpine sh -c "cd /data && tar xzf /backup/stamper-backup.tgz"
```

> **หมายเหตุ:** ควรเก็บไฟล์ backup ไว้บน drive อื่นหรือ shared drive ไม่ใช่โฟลเดอร์เดียวกับโปรเจกต์

8. การปรับค่าผ่านไฟล์ .env

สร้างไฟล์ชื่อ `.env` ในโฟลเดอร์โปรเจกต์ (ถ้ายังไม่มีให้สร้างใหม่เอง) เพื่อเปลี่ยนพอร์ตหรือค่าต่าง ๆ ของระบบ:

```
FRONTEND_PORT=8090
BACKEND_DEBUG_PORT=9012
PREVIEW_DPI=110
MAX_PDF_MB=50
MAX_COPIES=200
```

ตัวแปร	ความหมาย	ค่าเริ่มต้น
FRONTEND_PORT	พอร์ตของหน้าเว็บที่ทีมใช้งาน (เปลี่ยนถ้าชนกับโปรแกรมอื่น)	8090
BACKEND_DEBUG_PORT	พอร์ตเอกสาร API (/docs) — เข้าถึงจากเครื่องที่ติดตั้งเท่านั้น	9012
PREVIEW_DPI	ความละเอียดของภาพตัวอย่างในหน้าเว็บ (110–200)	110
MAX_PDF_MB	ขนาดไฟล์ PDF สูงสุดต่อไฟล์ (MB)	50
MAX_COPIES	จำนวนสำเนาสูงสุดต่อครั้ง	200

หลังแก้ไขไฟล์ .env ให้รัน `docker compose up -d` อีกครั้ง (ไม่ต้องใช้ --build)

9. แก้ปัญหาที่พบบ่อย

อาการ	สาเหตุและการแก้ไข
เปิด http://localhost:8090 ไม่ได้	Docker Desktop ยังไม่ทำงาน หรือคอนเทนเนอร์ยังไม่เริ่ม — รัน `docker compose ps` เพื่อตรวจสถานะ ถ้ายังไม่ได้ให้รัน `docker compose up -d --build` แล้วดู log ที่ `docker compose logs backend`
error เรื่องพอร์ต 8090 ถูกใช้งานอยู่แล้ว	เปลี่ยน `FRONTEND_PORT=8091` ในไฟล์ .env แล้วรัน `docker compose up -d`
build ไม่ผ่าน หรือดาวน์โหลดช้ามาก	อินเทอร์เน็ตช้าหรือถูกจำกัด — ลองใหม่อีกครั้ง หรือตั้ง proxy ที่ Docker Desktop → Settings → Resources → Proxies
คำสั่ง docker compose ไม่ทำงาน / WSL error	Docker Desktop เวอร์ชันเก่า — อัปเดตเป็นเวอร์ชันล่าสุด รัน `wsl --update` และตรวจว่าเปิด virtualization ใน BIOS แล้ว
เครื่องอื่นเข้าผ่าน LAN ไม่ได้	ยังไม่ได้เปิด firewall ตามข้อ 5, อยู่คนละวง LAN กับเครื่องที่ติดตั้ง, หรือ IP เปลี่ยน — ตรวจด้วยคำสั่ง ping ก่อน
อัปเดตแอปแล้วหน้าเว็บยังเป็นแบบเดิม	เบราว์เซอร์จำหน้าเก่าไว้ — กด **Ctrl+F5** (hard refresh)
อัปโหลดไฟล์แล้ว preview ไม่แสดง	ตรวจ `docker compose ps` ว่า backend สถานะ Up (healthy) แล้วดู log ตามข้อแรก
ข้อมูลงานเดิมหายไป	เกิดจากการใช้ `down -v` — กู้คืนได้จากไฟล์ stamper-backup.tgz ตามข้อ 7 เท่านั้น

10. รายการตรวจรับหลังติดตั้ง

- ☐ Docker Desktop แสดง “Engine running”
- ☐ `docker compose ps` ทั้ง backend และ frontend อยู่ในสถานะ Up (healthy)
- ☐ เปิด `http://localhost:8090` แล้วเห็นหน้าจอแอป
- ☐ อัปโหลดโลโก้และไฟล์ PDF ทดลอง แล้วเห็น watermark ตรงตามที่ตั้งไว้ใน preview
- ☐ กด Stamp แล้วดาวน์โหลดได้ไฟล์ PDF/ZIP ครบถ้วนถูกต้อง (ชื่อไฟล์ภาษาไทยใช้ได้)
- ☐ บันทึกงาน (preset) แล้ว reload หน้าเว็บ — ชื่องานยังคงอยู่
- ☐ (กรณีใช้ทั้งทีม) เครื่องอื่นเปิดผ่าน `http://<ip>:8090` ได้
- ☐ จัดตารางสำรองข้อมูล (backup volume) เป็นประจำ

—— จบคู่มือ —— หากแก้ตามนี้แล้วยังไม่หาย กรุณาบันทึกภาพหน้าจอที่แสดง error พร้อมผลของคำสั่ง `docker compose logs backend` ส่งให้ทีมพัฒนาระบบ ——